3. PBH 메커니즘 상세 분석

3.1 PBH 개념과 동작 원리 설명

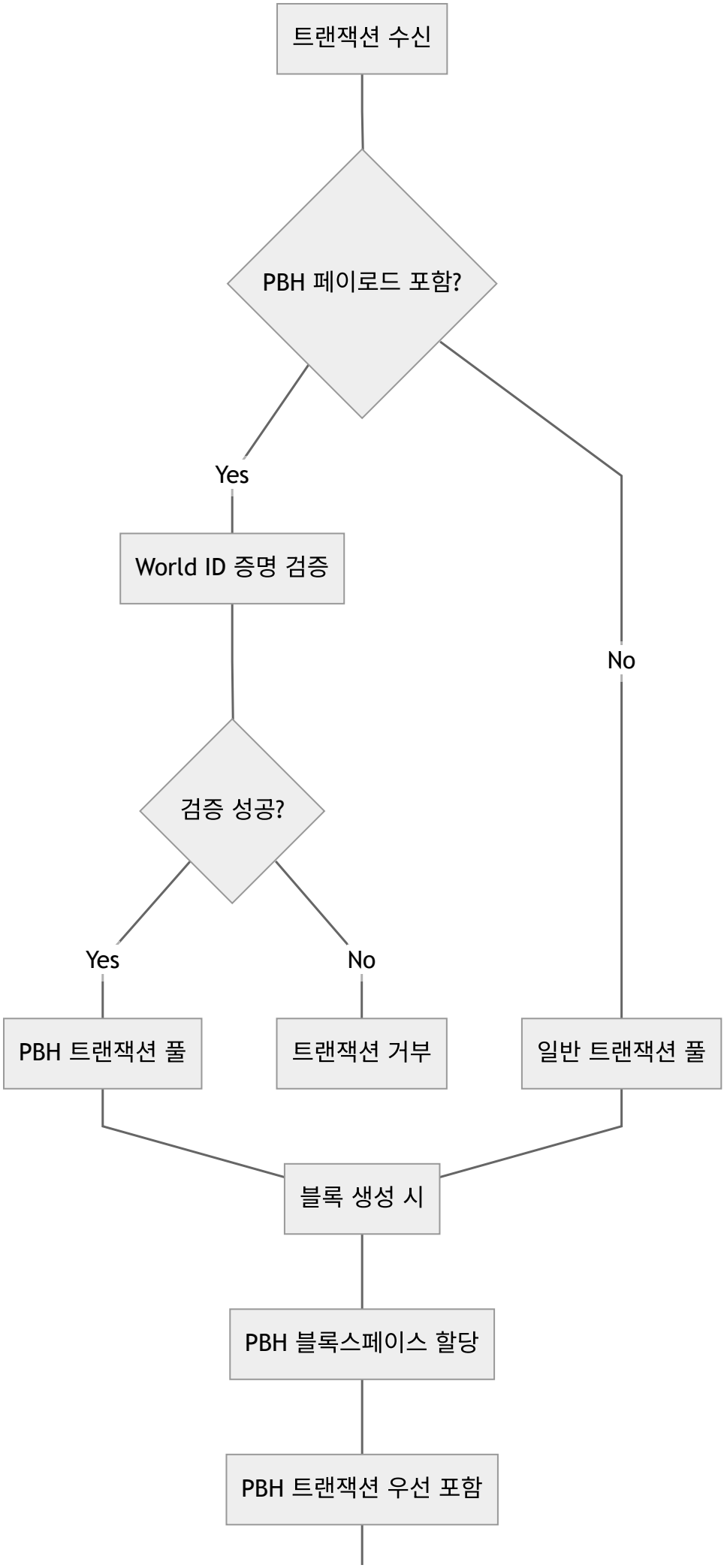
3.1.1 Priority Blockspace for Humans의 핵심 개념

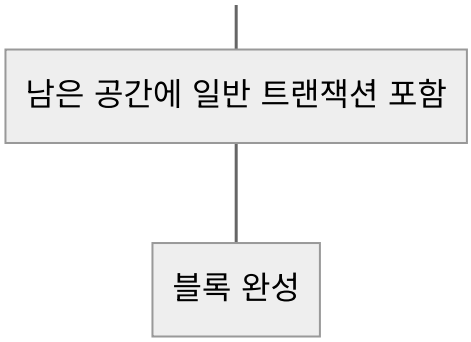
PBH(Priority Blockspace for Humans)는 World Chain의 핵심 혁신 기술로, 검증된 World ID 보유자에게 블록 내 우선순위를 부여하는 새로운 트랜잭션 순서 정책입니다. 이는 기존 블록체인에서 가스비 기반으로만 트랜잭션을 정렬하던 방식에서 벗어나, **인간 신원 검증**을 기반으로 한 공정한 블록스페이스 할당 시스템을 도입합니다.

핵심 원리

PBH는 다음과 같은 핵심 원리를 바탕으로 작동합니다:

1. **신원 기반 우선순위**: World ID 증명을 통해 실제 인간 사용자임을 확인
2. **블록스페이스 예약**: 각 블록에서 일정 비율을 PBH 트랜잭션을 위해 예약
3. **공정한 할당**: 검증된 사용자들 간에는 가스비 기준으로 공정하게 정렬
4. **시스템 호환성**: 남은 공간은 일반 트랜잭션으로 채워 기존 시스템과 호환성 유지


트랜잭션 수신
PBH 페이로드 포함?
Yes
No
World ID 증명 검증
검증 성공?
Yes
No
PBH 트랜잭션 풀
트랜잭션 거부
일반 트랜잭션 풀
블록 생성 시
PBH 블록스페이스 할당
PBH 트랜잭션 우선 포함



3.1.2 기존 트랜잭션 순서 정책과의 차이점 분석

기존 블록체인의 트랜잭션 정렬 방식

전통적인 블록체인에서는 **가스비 기반 경매 시스템**을 사용합니다:

```
기존 방식: 가스비 우선 정렬

┌─────────────────────────────────┐
│ 높은 가스비 → 높은 우선순위 │
│ 낮은 가스비 → 낮은 우선순위 │
│ │
│ 문제점: │
│ • 봇들의 가스비 경쟁 심화 │
│ • 일반 사용자 소외 │
│ • 네트워크 혼잡 시 접근성 저하 │
└─────────────────────────────────┘
```

World Chain의 PBH 정렬 방식

PBH는 **신원 검증 + 가스비 조합** 방식을 사용합니다:

```
PBH 방식: 신원 우선 + 가스비 보조

┌─────────────────────────────────┐
```

```
│ 1순위: World ID 검증 여부 │
│ 2순위: 가스비 (검증된 사용자 내) │
│ │
│ 장점: │
│ • 실제 사용자 우선 보장 │
│ • 예측 가능한 트랜잭션 비용 │
│ • 네트워크 혼잡 시에도 접근성 │
└──────────────────────────────────┘
```

구체적인 차이점 비교

구분	기존 방식	PBH 방식
정렬 기준	가스비만	World ID 검증 + 가스비
우선순위	높은 가스비 = 높은 우선순위	검증된 인간 = 최고 우선순위
공정성	경제적 능력에 의존	인간 신원에 기반
접근성	가스비 경쟁으로 제한적	검증된 사용자에게 보장
봇 대응	봇이 높은 가스비로 우선권 획득	봇은 PBH 블록스페이스 접근 불가

3.1.3 PBH 블록스페이스 용량과 예약 정책 설명

블록스페이스 용량 설정

World Chain Builder는 `verified_blockspace_capacity` 설정을 통해 PBH 전용 블록스페이스를 관리합니다:

```
// PBH 전용 가스 한도 계산
```

```
let verified_gas_limit = (self.verified_blockspace_capacity as u64 * gas_limit)


// 예시: 블록 가스 한도가 30,000,000이고 PBH 용량이 40%인 경우

// PBH 전용 가스 한도 = 12,000,000 가스
```

블록스페이스 예약 정책

PBH 블록스페이스 예약은 다음과 같은 정책을 따릅니다:

1. **고정 비율 예약**: 각 블록에서 설정된 비율만큼 PBH 트랜잭션을 위해 예약
2. **유연한 할당**: PBH 트랜잭션이 부족한 경우 일반 트랜잭션으로 채움
3. **오버플로우 처리**: PBH 트랜잭션이 용량을 초과하는 경우 다음 블록으로 이월

```
graph LR

A[블록 가스 한도<br/>30M 가스] --> B[PBH 예약<br/>12M 가스<br/>40%]

A --> C[일반 트랜잭션<br/>18M 가스<br/>60%]

B --> D{PBH 트랜잭션<br/>충분한가?}

D -->|Yes| E[PBH로 채움]

D -->|No| F[일반 트랜잭션으로<br/>나머지 채움]

C --> G[가스비 순으로<br/>일반 트랜잭션 포함]
```

실제 블록 구성 예시

```
블록 구성 예시 (가스 한도: 30,000,000, PBH 용량: 40%)
```

```
PBH 블록스페이스 (12,000,000 가스):

┌─────────────────────────────────┐

| 🟢 PBH Tx 1: 2,000,000 가스 |

| 🟢 PBH Tx 2: 1,500,000 가스 |

| 🟢 PBH Tx 3: 3,000,000 가스 |

| 🟢 PBH Tx 4: 2,500,000 가스 |

| 🟢 PBH Tx 5: 1,800,000 가스 |

| 🟢 PBH Tx 6: 1,200,000 가스 |

| 총 사용: 12,000,000 가스 |

└─────────────────────────────────┘


일반 블록스페이스 (18,000,000 가스):

┌─────────────────────────────────┐

| 일반 Tx 1: 500,000 가스 |

| 일반 Tx 2: 800,000 가스 |

| 일반 Tx 3: 1,200,000 가스 |

| ... (가스비 순으로 정렬) |

| 총 사용: 18,000,000 가스 |

└─────────────────────────────────┘
```

동적 할당 메커니즘

PBH 시스템은 네트워크 상황에 따라 동적으로 블록스페이스를 할당합니다:

시나리오 1: PBH 트랜잭션 부족

```
PBH 예약: 12M 가스

실제 PBH: 8M 가스

→ 남은 4M 가스를 일반 트랜잭션으로 채움

→ 총 일반 트랜잭션 공간: 22M 가스
```

시나리오 2: PBH 트랜잭션 과다

```
PBH 예약: 12M 가스

대기 중인 PBH: 20M 가스

→ 12M 가스만 현재 블록에 포함

→ 8M 가스 상당의 PBH 트랜잭션은 다음 블록으로 이월
```

공정성 보장 메커니즘

PBH 시스템은 다음과 같은 방식으로 공정성을 보장합니다:

1. **월별 사용량 제한**: External Nullifier를 통한 월별 PBH 트랜잭션 수 제한
2. **Nonce 기반 중복 방지**: 동일한 nonce 재사용 방지로 어뷰징 차단
3. **가스비 기반 내부 정렬**: PBH 트랜잭션 간에는 여전히 가스비로 정렬
4. **일반 사용자 보장**: 최소 60%의 블록스페이스는 일반 트랜잭션을 위해 보장

이러한 PBH 메커니즘을 통해 World Chain은 "봇이 혼잡을 만드는 곳에서 인간을 위한 고속도로"라는 비전을 실현하며, 실제 사용자들에게 더 공정하고 예측 가능한 블록체인 경험을 제공합니다.

3.2 PBH 페이로드 구조 분석

3.2.1 PBH 페이로드 개요

PBH 페이로드는 World Chain에서 PBH 트랜잭션의 핵심 구성 요소로, World ID 증명과 관련된 모든 필수 데이터를 포함합니다. 이 페이로드는 RLP(Recursive Length Prefix) 인코딩을 사용하여 직렬화되며, 다음과 같은 4개의 주요 필드로 구성됩니다:

```
#[derive(Default, Clone, Debug, RlpEncodable, RlpDecodable, PartialEq, Eq)]
pub struct PBHPayload {
pub external_nullifier: ExternalNullifier,
pub nullifier_hash: Field,
pub root: Field,
pub proof: Proof,
}
```

3.2.2 External Nullifier 필드 상세 분석

구조와 인코딩 방식

`external_nullifier` 는 PBH 트랜잭션의 고유성과 월별 사용량 제한을 보장하는 핵심 식별자입니다. 이 필드는 다음과 같은 구조로 인코딩됩니다:

```
External Nullifier 인코딩 구조 (256비트):
┌──────────────────────────────────────────────────────────────┐
| Bits 255-48: Empty (208 bits) |
├──────────────────────────────────────────────────────────────┤
| Bits 47-32: Year (16 bits) | 예: 2024 = 0x07E8 |
├──────────────────────────────────────────────────────────────┤
| Bits 31-24: Month (8 bits) | 예: 12월 = 0x0C |
├──────────────────────────────────────────────────────────────┤
```

```
| Bits 23-8: Nonce (16 bits) | 예: 5번째 = 0x0005 |
├──────────────────────────────────────────────────────────────┤
| Bits 7-0: Version (8 bits) | 현재 버전 = 0x01 |
└──────────────────────────────────────────────────────────────┘
```

인코딩 구현

```rust
impl From<ExternalNullifier> for EncodedExternalNullifier {
fn from(e: ExternalNullifier) -> Self {
EncodedExternalNullifier(U256::from(
(e.year as u64) << 32
| (e.month as u64) << 24
| (e.nonce as u64) << 8
| e.version as u64,
))
}
}
```

실제 사용 예시

```rust
// 2024년 12월, 5번째 PBH 트랜잭션
let external_nullifier = ExternalNullifier::v1(12, 2024, 5);

// 인코딩 결과: 0x0000000000000000000000000000000000000000007E80C0501
```

```
// 분해:
// - Year: 0x07E8 (2024)
// - Month: 0x0C (12월)
// - Nonce: 0x0005 (5번째)
// - Version: 0x01 (v1)
```

Date Marker와의 연동

External Nullifier는 `DateMarker` 구조체와 연동하여 월별 기간을 관리합니다:

```rust
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct DateMarker {
pub year: i32,
pub month: u32,
}

impl ExternalNullifier {
pub fn date_marker(&self) -> DateMarker {
DateMarker::new(self.year as i32, self.month as u32)
}
}
```

3.2.3 Nullifier Hash 필드 분석

역할과 목적

`nullifier_hash` 는 Semaphore 프로토콜의 핵심 구성 요소로, 다음과 같은 역할을 수행합니다:

1. **중복 방지**: 동일한 external nullifier로 여러 번 증명 생성 방지
2. **프라이버시 보장**: 실제 신원을 노출하지 않으면서 고유성 보장
3. **추적 불가능성**: 서로 다른 nullifier hash 간 연관성 파악 불가

생성 방식

Nullifier hash는 다음과 같은 방식으로 생성됩니다:

```
// Semaphore 프로토콜에서의 nullifier hash 생성
let nullifier_hash = semaphore_rs::protocol::generate_nullifier_hash(
&identity, // 사용자의 World ID 신원
external_nullifier_hash // 인코딩된 external nullifier
);
```

검증 과정

World Chain Builder는 nullifier hash를 사용하여 중복 사용을 방지합니다:

```
// 중복 nullifier hash 검사
if used_nullifier_hashes.contains(&pbh_payload.nullifier_hash) {
return Err(PBHValidationError::DuplicateNullifierHash);
}
```

3.2.4 Root 필드 분석

Merkle Root의 역할

`root` 필드는 World ID 집합을 나타내는 Merkle Tree의 루트 해시입니다:

1. **포함 증명**: 사용자가 검증된 World ID 집합에 포함되어 있음을 증명
2. **집합 무결성**: 전체 World ID 집합의 상태를 압축적으로 표현
3. **효율적 검증**: 전체 집합을 다운로드하지 않고도 포함 여부 확인 가능

유효한 Root 검증

World Chain Builder는 유효한 root 목록을 유지하고 검증합니다:

```
pub fn validate_root(&self, valid_roots: &[Field]) -> Result<(), PBHValidation

if !valid_roots.contains(&self.root) {

return Err(PBHValidationError::InvalidRoot);

}

Ok(())

}
```

Root 업데이트 메커니즘

World ID 시스템에서 새로운 사용자가 추가되거나 기존 사용자가 제거될 때마다 새로운 root가 생성됩니다:

```
Merkle Tree 업데이트 과정:

┌─────────────────────────────────┐

| 기존 Root: 0xabc123... |

| 새 사용자 추가 |

| 새 Root: 0xdef456... |

| |
```

```
| Builder는 두 root 모두 유효하게 |
| 인정 (일정 기간 동안) |
└──────────────────────────────┘
```

3.2.5 Proof 필드 분석

Semaphore 증명 구조

`proof` 필드는 Semaphore 영지식 증명을 포함하며, 다음과 같은 구조를 가집니다:

```rust
pub struct Proof(pub semaphore_rs::protocol::Proof);

// Semaphore 증명의 내부 구조
pub struct Proof(
(U256, U256), // a (2개 요소)
([U256, U256], [U256, U256]), // b (2x2 행렬)
(U256, U256), // c (2개 요소)
);
```

압축된 증명 지원

World Chain은 효율성을 위해 압축된 증명을 지원합니다:

```rust
// 압축된 증명 감지 및 처리
let flat = self.proof.0.flatten();
let proof = if (flat[4] | flat[5] | flat[6] | flat[7]).is_zero() {
```

```
// 증명이 압축됨 - 압축 해제 필요
let compressed_flat = [flat[0], flat[1], flat[2], flat[3]];
let compressed_proof =
semaphore_rs_proof::compression::CompressedProof::from_flat(compressed_flat);
&semaphore_rs_proof::compression::decompress_proof(compressed_proof)
.ok_or(PBHValidationError::InvalidProof)?
} else {
// 일반 증명
&self.proof.0
};
```

증명 검증 과정

Semaphore 증명은 다음과 같은 공개 입력들과 함께 검증됩니다:

```
// 증명 검증
if verify_proof(
self.root, // Merkle root
self.nullifier_hash, // Nullifier hash
signal, // 트랜잭션 해시
EncodedExternalNullifier::from(self.external_nullifier).0, // External nullifier
proof, // 증명
TREE_DEPTH, // Tree 깊이 (30)
)? {
Ok(())
```

```
} else {
Err(PBHValidationError::InvalidProof)
}
```

3.2.6 RLP 인코딩 방식과 데이터 구조

RLP 인코딩 개요

PBH 페이로드는 Ethereum의 표준 RLP(Recursive Length Prefix) 인코딩을 사용합니다:

```
#[derive(Default, Clone, Debug, RlpEncodable, RlpDecodable, PartialEq, Eq)]
pub struct PBHPayload {
pub external_nullifier: ExternalNullifier, // RLP 인코딩 가능
pub nullifier_hash: Field, // U256으로 인코딩
pub root: Field, // U256으로 인코딩
pub proof: Proof, // 256바이트 배열로 인코딩
}
```

각 필드별 인코딩 방식

1. External Nullifier 인코딩

```
impl Encodable for ExternalNullifier {
fn encode(&self, out: &mut dyn alloy_rlp::BufMut) {
EncodedExternalNullifier::from(*self).encode(out);
}
```

```
}
```

2. Field (U256) 인코딩

```
// nullifier_hash와 root는 모두 U256 타입으로 직접 RLP 인코딩
// Alloy의 기본 U256 RLP 인코딩 사용
```

3. Proof 인코딩

```
const LEN: usize = 256; // 256바이트 고정 길이
pub type ProofBytes = [u8; LEN];

impl Encodable for Proof {
fn encode(&self, out: &mut dyn alloy_rlp::BufMut) {
let PackedProof(bytes) = self.0.into();
bytes.encode(out); // 256바이트 배열로 인코딩
}

fn length(&self) -> usize {
LEN + 3 // 256바이트 + RLP 오버헤드
}
}
```

완전한 RLP 구조

```
PBH 페이로드 RLP 구조:

┌─────────────────────────────────────────────────────────────┐
| RLP List Header |
├─────────────────────────────────────────────────────────────┤
| External Nullifier (32 bytes) |
| - RLP 인코딩된 U256 값 |
├─────────────────────────────────────────────────────────────┤
| Nullifier Hash (32 bytes) |
| - RLP 인코딩된 U256 값 |
├─────────────────────────────────────────────────────────────┤
| Root (32 bytes) |
| - RLP 인코딩된 U256 값 |
├─────────────────────────────────────────────────────────────┤
| Proof (256 bytes) |
| - RLP 인코딩된 바이트 배열 |
└─────────────────────────────────────────────────────────────┘

총 크기: ~352 bytes + RLP 오버헤드
```

인코딩/디코딩 테스트

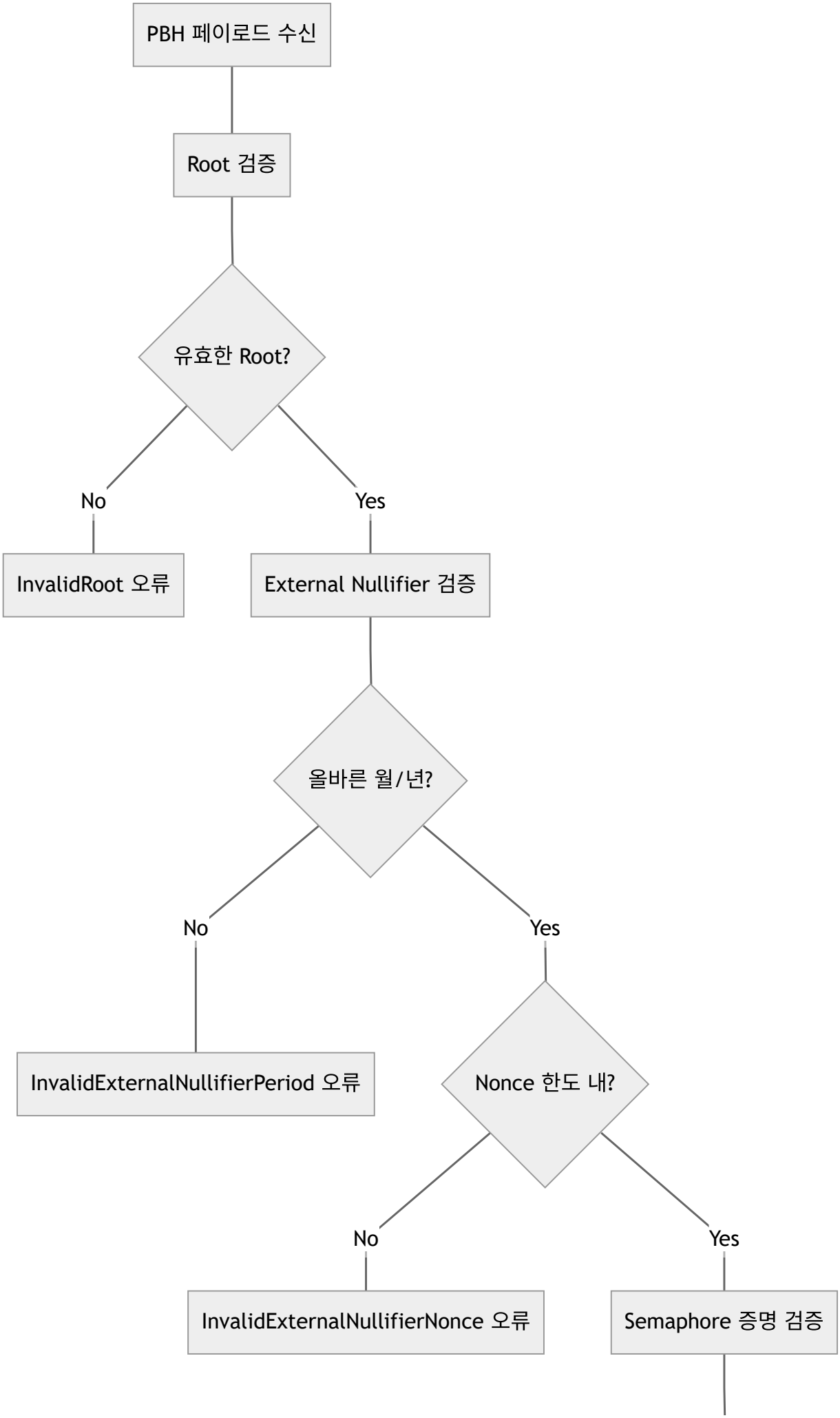
```rust
#[test]
fn encode_decode() {
let proof = Proof(semaphore_rs::protocol::Proof(
```

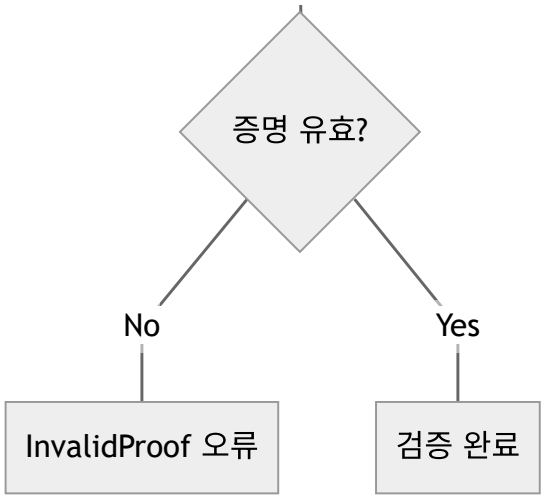
```
(U256::from(1u64), U256::from(2u64)),
(
[U256::from(3u64), U256::from(4u64)],
[U256::from(5u64), U256::from(6u64)],
),
(U256::from(7u64), U256::from(8u64)),
));
let pbh_payload = PBHPayload {
external_nullifier: ExternalNullifier::v1(1, 2024, 11),
nullifier_hash: Field::from(10u64),
root: Field::from(12u64),
proof,
};

// 인코딩
let mut out = vec![];
pbh_payload.encode(&mut out);
// 디코딩
let decoded = PBHPayload::decode(&mut out.as_slice()).unwrap();
assert_eq!(pbh_payload, decoded);
}
```

3.2.7 페이로드 검증 프로세스

PBH 페이로드는 다음과 같은 단계별 검증을 거칩니다:


PBH 페이로드 수신
Root 검증
유효한 Root?
No
Yes
InvalidRoot 오류
External Nullifier 검증
올바른 월/년?
No
Yes
InvalidExternalNullifierPeriod 오류
Nonce 한도 내?
No
Yes
InvalidExternalNullifierNonce 오류
Semaphore 증명 검증



이러한 상세한 페이로드 구조와 검증 과정을 통해 World Chain은 안전하고 효율적인 PBH 메커니즘을 구현하여, 검증된 사용자들에게 우선순위 블록스페이스를 제공합니다.